

14Tanggal publikasi xxxx 00,0000, tanggal versi saat ini xxxx 00,0000.

Pengenal Objek Digital 10.1109/ACCESS.2022.Nomor Doi

MSNws: Pengontrol SDN Berbasis Layanan Mikro menggunakan Protokol WebSocket untuk Menstabilkan Jaringan Penggunaan dan Kinerja dalam Pengirisan Multi-Penyewa

Mochamad Rafli Hadiana, Muhammad Litfan Rahmansyah, Syekh Maulana Wijaya, Salman Alfarizi NB, Ridha Muldina Negara dan Sofia Naning Hertiana

Jurusan Teknik Elektro, Telkom University, 40257 Indonesia

Penulis koresponden: Mochamad Rafli Hadiana (email: raflihadiana@student.telkomuniversity.ac.id).

Pekerjaan ini sebagian didukung oleh infrastruktur penelitian Adaptive Network Laboratory yang didanai oleh Telkom University.

ABSTRAK Dengan semakin maraknya teknologi cloud, seperti orkestrasi container pada jaringan seluler generasi kelima (5G), diperlukan fleksibilitas yang lebih besar dalam proses manajemen jaringan menuju jaringan backend yang kompleks. Software Defined Networking (SDN) adalah teknologi lain yang banyak digunakan yang memungkinkan kontrol jaringan lebih dinamis terhadap fungsi jaringan virtual (VNF), terutama dalam skenario jaringan seluler. Menilai arsitektur dan protokol komunikasi yang mengontrol fungsi switch sangat penting untuk mengoptimalkan kinerja jaringan. Karya ini menyajikan Microservices-Based SDN Controller WebSocket Protocol (MSNws), sebuah SDN Controller yang mengimplementasikan arsitektur modern, yang berfokus pada integrasi ke WebSocket sebagai protokol komunikasi utama dan menerapkan middleware yang menyediakan kontrol aliran dan rute. Pengontrol yang diusulkan menggunakan teknologi sumber terbuka untuk memberikan alternatif berbiaya rendah sambil menekankan fungsionalitas, manajemen, dan orkestrasi cloud. Makalah ini menjelaskan kemampuan pengontrol yang diusulkan untuk menstabilkan kinerja jaringan dalam skenario implementasi pemotongan jaringan dan keuntungannya dalam mengurangi pemanfaatan sumber daya komputasi.

ISTILAH INDEKS Software Defined Networking (SDN), Kontainerisasi, Konsumsi CPU, Layanan Mikro Pengontrol, Pemotongan Jaringan.

I. PENDAHULUAN

Pada jaringan seluler generasi kelima (5G), Software-Defined Network (SDN) merupakan salah satu faktor penting dalam pengelolaan jaringan terpusat. SDN adalah paradigma jaringan yang memecah infrastruktur jaringan tradisional dengan memisahkan fungsi jaringan dari router dan switch yang mendasari lalu lintas [1]–[3]. Ini membuat peralihan jaringan lebih mudah untuk diteruskan, dan fungsi kontrol menentukan kebijakan dan konfigurasi jaringan. Fungsi kontrol lainnya adalah agen pusat untuk mengimplementasikan persyaratan jaringan aplikasi dan melakukan beberapa fungsi dari aplikasi perangkat lunak, seperti analisis dan pengukuran Quality of Service (QoS), routing lanjutan, dan kontrol lalu lintas [4]. Dalam kebanyakan kasus, ia berkomunikasi dengan perangkat penerus yang bertanggung jawab untuk meneruskan dan memanipulasi paket menggunakan OpenFlow sebagai protokol komunikasi jaringan [3].

Fleksibilitas yang ditujukan oleh fungsi kontrol terpusat saat ini masih menimbulkan toleransi kesalahan, kendala latensi, dan masalah penyeimbangan beban karena pengontrol yang ada tidak berfungsi.

diimplementasikan sebagai satu kesatuan monolitik. Bahkan jika digunakan sebagai sistem pengontrol terdistribusi, beberapa subfungsi belum tentu digunakan untuk mengontrol jaringan [5]. Masalah utama dengan arsitektur perangkat lunak monolitik adalah kebutuhan untuk memberikan lebih banyak fleksibilitas kepada administrator jaringan untuk memilih komponen atau fungsi tertentu untuk mengimplementasikan kebutuhan khusus mereka untuk skenario yang berbeda [6].

Namun, dengan bangkitnya arsitektur modern dalam rekayasa perangkat lunak, pengontrol SDN dapat didekomposisi dan diimplementasikan menjadi komponen mikro yang terdiri dari sub-fungsionalis dan masih berbagi layanan jaringan beban yang sama, sehingga menciptakan sistem yang tangguh terhadap kegagalan [5]. Arsitektur layanan mikro merupakan bagian penting dari strategi untuk membuat pengontrol arsitektur tradisional menyediakan penerapan yang lebih fleksibel. Teknologi container seperti Docker dan Kubernetes dirancang untuk menyediakan platform isolasi terstandar yang membantu pengembangan aplikasi dengan menyederhanakan dan mempercepat proses penerapan [7]–[9]. Keuntungan utama dari layanan mikro adalah menangani operasional

tantangan infrastruktur dengan membatasi dan menyepakati bagaimana komponen perangkat lunak diterapkan dalam container. Ini memfasilitasi pemeliharaan dan keandalan sistem perangkat lunak yang dikelola oleh orkestrator.

Oleh karena itu, beberapa studi penelitian telah dimulai untuk mengimplementasikan pengontrol SDN untuk layanan mikro. ONOS menggunakan arsitektur berbasis layanan mikro yang dirancang untuk skenario pusat data cloud dan menggunakan Panggilan Prosedur Jarak Jauh Google (gRPC) sebagai komunikasi antar-fungsi, namun masih terbatas pada kurangnya implementasi dalam kepatuhan 5G karena masih dalam tahap awal. produk dan membutuhkan lebih banyak pekerjaan untuk menyediakan kerangka bermain [5], [10]. Selain itu, studi penelitian lain yang disebut Microservice SDN-controller (MSN) menggabungkan Pengontrol Ryu SDN dan REST API sebagai protokol komunikasi utama yang dirancang khusus untuk penerapan 5G RAN Edge generasi berikutnya. Namun, ini tidak menunjukkan implementasi skenario jaringan multi-penyewa dan evaluasi kinerja pengontrol [5], [11]. Di masa depan, salah satu fitur yang diantisipasi dari aplikasi 5G adalah network slicing, yang berpotensi menyediakan konektivitas yang disesuaikan. Konektivitas yang disesuaikan ini terbukti sangat menguntungkan bagi berbagai industri, karena menawarkan pendekatan yang lebih cerdas dalam membagi jaringan untuk melayani layanan atau sektor bisnis tertentu. Oleh karena itu, paradigma pemotongan jaringan yang sesuai untuk memenuhi tujuan ini mencakup multi-tenancy [12]–[15]. Ini juga merupakan gagasan multi-tenancy

memungkinkan beberapa operator jaringan virtual untuk berbagi infrastruktur fisik [16], [17].

Makalah ini menjelaskan pengontrol SDN layanan mikro yang diusulkan dengan protokol komunikasi WebSocket diimplementasikan dalam skenario jaringan multi-penyewa. Kontainer-orchestrator membuat penerapan aplikasi untuk membuat jaringan yang saling terhubung ke switch OpenFlow tervirtualisasi, pengontrol SDN, dan pengontrol proksi. Testbed dibangun dengan perangkat lunak sumber terbuka dan VNF untuk memfasilitasi penerapan dan interoperabilitas proses penelitian.

Makalah ini menyajikan dua kasus penggunaan, satu untuk pemotongan multi-penyewa dan satu lagi untuk pengujian pemanfaatan sumber daya. Kode sumber disediakan untuk memungkinkan replikasi kedua eksperimen ini.

Kontribusi makalah ini adalah sebagai berikut:

- Mengusulkan modifikasi protokol WebSocket untuk arsitektur pengontrol SDN berbasis layanan mikro untuk mengelola testbed.
- Makalah ini mengevaluasi berbagai arsitektur pengontrol SDN yang sesuai untuk memberikan kinerja jaringan dan pemanfaatan sumber daya yang lebih stabil kepada operator sesuai dengan meningkatnya jumlah ukuran lalu lintas latar belakang.
- Eksperimen dapat direproduksi, mengingat kode untuk menerapkan testbed dan dua aplikasi kasus penggunaan tersedia untuk umum.

Makalah ini disusun sebagai berikut. Bagian II menjelaskan pengontrol layanan mikro dalam akademi penelitian terbaru,

menekankan tujuan dan penerapan praktisnya untuk memberikan konteks pada pengujian MSNws. Bagian III menguraikan struktur logis dan atribut utama MSNws. Bagian IV menjelaskan dua kasus penggunaan yang dieksplorasi di mana teknik pemotongan jaringan diterapkan, dan pengujian stres dilakukan menggunakan testbed, beserta hasil yang diperoleh. Terakhir, Bagian V menjelaskan langkah selanjutnya dari terbitan terbuka dan menyimpulkan makalah.

II. PEKERJAAN TERKAIT: KONTROLER LAYANAN MIKRO

Implementasi pengontrol SDN berbasis layanan mikro difokuskan pada penguraian versi monolitiknya agar dapat diterapkan dalam arsitektur layanan mikro. Demikian penelitian ini mengacu pada studi yang ada tentang dekomposisi pengontrol SDN sebagai panduan untuk merancang testbed kami dengan trade-off minimal dalam jaringan multi-penyewa. Kami telah menganalisis dua studi penelitian yang ada, ONOS dan Ryu MSN, dan menyimpulkan bahwa dekomposisi Ryu MSN lebih fleksibel dan dapat diterapkan di pengujian kami.

ONOS adalah arsitektur ONOS generasi berikutnya, yang menganut desain layanan mikro. Namun pengontrol ini terfokus pada skenario pusat data cloud dan tidak memiliki fleksibilitas dalam mengubah protokol komunikasi [18].

Selanjutnya, protokol komunikasi default ONOS adalah gRPC, dan menurut penelitian terbaru [5], protokol ini memiliki latensi lebih tinggi dibandingkan WebSocket. Penelitian ini dimaksudkan untuk meminimalisir latensi antar microservice agar lebih dekat dengan pengontrol SDN monolitik yang tidak memiliki latensi karena tidak adanya komunikasi antar node di dalam pengontrol SDN monolitik itu sendiri.

Pada akhirnya MSN digunakan sebagai referensi utama, upaya yang ada dalam menguraikan pengontrol Ryu SDN menjadi arsitektur berbasis layanan mikro dengan memisahkan logika aplikasi SDN dari komponen dasar Ryu.

Itu ada di node middleware untuk mengkonversi format dari acara OpenFlow ke JSON dan, juga, karena ketidakmampuan logika aplikasi untuk memproses objek terkait.

Sedangkan komponen hanya bertanggung jawab menyediakan logika jaringan. Melalui pemanfaatan teknik dekomposisi ini, komponen logika aplikasi dapat didistribusikan sekaligus menghindari duplikasi komponen dasar, yaitu middleware [5], [19], [20].

Kami telah merancang testbed kami dengan metode dekomposisi penelitian MSN sebelumnya. Namun, MSNws, yang merupakan pengontrol yang diusulkan, menggunakan WebSocket sebagai protokol komunikasi untuk meminimalkan latensi, menyederhanakan basis kode logika, dan menyediakan kode sumber kepada publik untuk pengembangan di masa mendatang¹. Selain itu, penelitian ini menganalisis implementasi pengontrol MSNws pada jaringan multi-tenant dengan skenario network slicing. Selain itu, kami merancang logika jaringan untuk mengkonfigurasi topologi pohon lemak sebagai uji kasus implementasi.

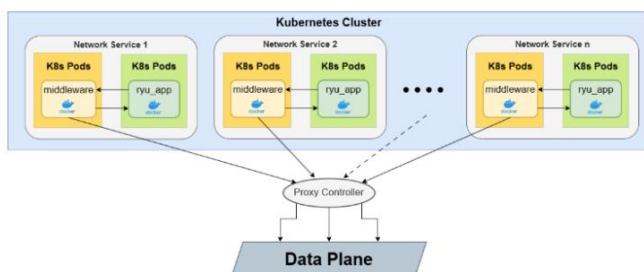
¹ <https://github.com/adaptivenetworklab/cloud-sdn>

AKU AKU AKU. MSNWS: KONTROLER SDN WEBSOCKET

Pengontrol SDN modifikasi yang baru diusulkan menggunakan Ryu sebagai kerangka utama. Arsitektur yang diusulkan didasarkan pada layanan mikro, seperti yang ditunjukkan pada Gambar 1. Dalam arsitektur ini, semua komponen dan aplikasi Ryu didekomposisi sebagai layanan mikro daripada digabungkan dalam fungsi lingkungan monolit. Penelitian ini memanfaatkan Mininet SDN dengan spesifikasi yang sesuai untuk mengelola jaringan multi-tenant yang kompleks. Integrasi dengan Pengontrol Ryu SDN, berdasarkan layanan mikro, memungkinkan pengembangan sistem kontrol multi-penyewa secara efektif. Tujuannya adalah mengelola sumber daya untuk meningkatkan kinerja dan skalabilitas jaringan secara efisien. Keuntungan Mininet adalah fleksibilitas bandwidthnya, yang memungkinkan penyesuaian dinamis berdasarkan penggunaan. Pendekatan ini memungkinkan eksplorasi tentang bagaimana pengontrol SDN berbasis layanan mikro beradaptasi terhadap perubahan dinamika jaringan, sehingga meningkatkan kemampuan manajemen sumber daya.

A. DESAIN KONTROLER LAYANAN MIKRO

Pekerjaan kami memanfaatkan kerangka pengontrol Ryu SDN, yang secara tradisional dianggap monolitik, sebagai elemen dasar untuk pengontrol layanan mikro kami. Komponen ini, disebut OpenFlow Controller WebSocket (ofctl_ws), berada di dalam lapisan middleware. Selain itu, kami telah memperkenalkan kemampuan baru yang berasal dari pengontrol monolitik ini, yang bertanggung jawab untuk mengelola logika jaringan dan mengonfigurasi jaringan Mininet kami. Kami dengan tepat menamakan kemampuan ini sebagai logika aplikasi. Logika aplikasi ditempatkan di layanan terpisah yang berbeda dari komponen pengontrol Ryu SDN dasar. Pilihan arsitektur ini memastikan pemisahan tanggung jawab yang jelas dan meningkatkan modularitas sistem kami.

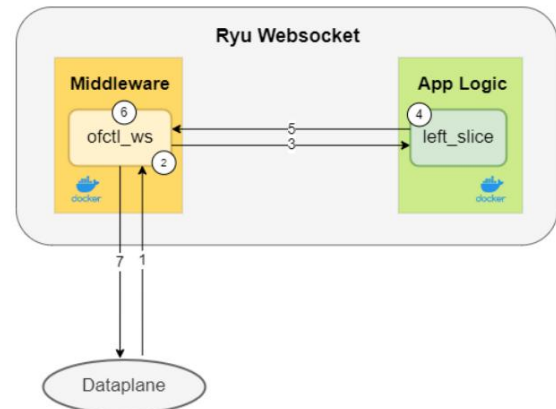


GAMBAR 1. Arsitektur pengontrol yang diusulkan.

Komponen dasar beroperasi sebagai antarmuka OpenFlow, mengelola penerimaan dan transmisi kejadian OpenFlow ke dan dari bidang data. Sebaliknya, logika aplikasi mendefinisikan logika konfigurasi untuk jaringan Mininet kami. Hasilnya, kami memisahkan fungsionalitas OpenFlow dari logika pemrosesan, mengalihkan beban pemrosesan logis ke layanan yang berbeda. Pemisahan strategis ini menghilangkan kebutuhan pemrosesan semua tugas dalam Ryu Base Controller, yang terletak di dalam lapisan middleware.

Desain seperti itu menumbuhkan modularitas dan meningkatkan efisiensi arsitektur sistem kami.

Untuk lebih presisi, arahkan perhatian ke Gambar 2, yang memberikan ilustrasi detail tentang dinamika komunikasi antara layanan mikro dan bidang data.



GAMBAR 2. Alur komunikasi pengontrol layanan mikro kami.

Prosesnya dimulai dengan bidang data yang mentransmisikan OpenFlow Packet-In (opf_packetin) ke middleware menggunakan protokol OpenFlow. Setelah berada di dalam middleware, opf_packetin mengalami transformasi menjadi format JSON, yang dinotasikan sebagai json_packetin. Selanjutnya, middleware menggunakan protokol WebSocket untuk meneruskan json_packetin ke logika aplikasi. Json_packetin yang diterima diproses dengan cermat dalam komponen logika aplikasi, menerapkan algoritma topologi untuk menghasilkan json_packetout. Json_packetout yang dihasilkan ini kemudian diteruskan kembali ke middleware melalui protokol WebSocket. Setelah masuk kembali ke middleware, json_packetout dikembalikan ke bentuk aslinya sebagai OpenFlow Packet-Out (opf_packetout). Terakhir, middleware kembali menggunakan protokol OpenFlow untuk mengirimkan opf_packetout ke bidang data, mengakhiri siklus pertukaran data.

Untuk memfasilitasi komunikasi antara komponen dasar dan logika aplikasi, yang berada di layanan terpisah, kami membuat koneksi melalui Antarmuka Pemrograman Aplikasi (API). Seperti yang disorot dalam makalah [5], protokol komunikasi dengan latensi terendah untuk kasus VNF adalah WebSocket, itulah pilihan kami. Di jaringan multi-penyewa, tempat kami mengelola bidang data, kami menggunakan Flowvisor sebagai pengontrol proksi untuk memfasilitasi pemotongan jaringan. Ini adalah komponen virtualisasi jaringan yang penting, diposisikan secara strategis antara pengontrol SDN utama dan bidang data. Ia memanipulasi paket OpenFlow, membangun jaringan virtual terisolasi dalam infrastruktur bidang data yang lebih luas. Masing-masing jaringan terisolasi ini mewakili potongan jaringan individual, yang dioptimalkan secara cermat untuk melayani layanan tertentu. Akibatnya, setiap peristiwa Packet-Out atau FlowMod yang ditujukan untuk bidang data menjalani pemrosesan awal oleh Flowvisor. Langkah ini mencakup pemfilteran untuk mencegah komunikasi yang tidak diinginkan antar irisan jaringan, memastikan integritas dan isolasi operasi setiap irisan [21].

B.ORKESTRASI VNF

Kontainerisasi adalah teknologi yang menggabungkan aplikasi, perpustakaan sistem, dan dependensi terkait ke dalam penerapan terisolasi yang terorganisir dan dikemas [22]. Penerapan yang dibangun dan dijalankan sebagai container dapat dieksekusi, mudah dimigrasi, dan dapat diskalakan. Hal ini membantu meningkatnya permintaan aplikasi atau fungsi yang diminta, menambah jumlah kontainer untuk menangani lalu lintas akan cukup untuk menangani lalu lintas. Dalam arsitektur Pengontrol SDN modifikasi yang kami usulkan, kami menggunakan Docker untuk teknologi kontainerisasi karena merupakan pengelola container yang ringan, sumber terbuka, dan mudah ditangani [23].

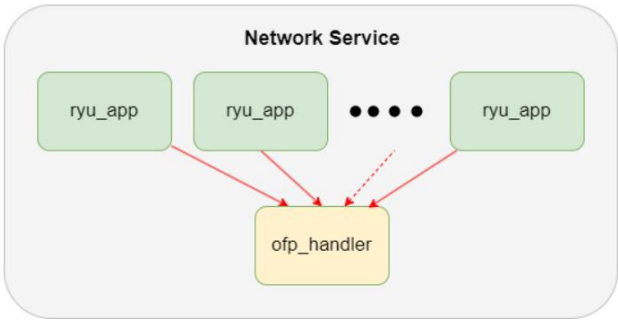
Dengan spekulasi mengenai jaringan lingkungan yang luas yang akan menangani banyak container, kami memerlukan orkestrator container untuk mengelola dan mengatur penerapan container Docker kami. Oleh karena itu, kami akan menggunakan Kubernetes sebagai container-orchestrator karena beberapa hak istimewa yang diberikan, seperti kemudahan penerapan perangkat lunak dan layanan, otomatisasi penerapan perangkat lunak, ketersediaan provisi perangkat lunak yang tinggi, dan spesifikasi batas sumber daya [24]. Kubernetes akan membantu pengelolaan container agar mudah terhubung ke pengontrol proxy eksternal atau data plane tanpa perlu memasukkan sumber IP Address secara manual setiap kali deployment dibuat. Lalu lintas antar host Kubernetes akan dikelola oleh salah satu Container Network Interface (CNI) yang didukung, Flannel. Flannel tidak mengontrol bagaimana container dihubungkan ke host [25], sehingga setiap komunikasi antar container (pod) akan diteruskan melalui Flannel. CNI ini juga akan membantu menghubungkan setiap sub-fungsi Pengontrol SDN dan aplikasi VNF yang di-deploy, karena setiap pod didistribusikan di antara Cluster Kubernetes pada host yang berbeda.

Di SDN, perangkat penerus seperti switch akan memproses paket masuk dan meminta aliran dari pengontrol sehingga paket akan diteruskan ke rute tertentu yang sudah dikonfigurasi di pengontrol. Ada beberapa tipe kejadian atau pesan OpenFlow untuk komunikasi antara pengontrol SDN dan perangkat penerusan. Tipe pertama, dikenal sebagai Event Packet-In, terjadi ketika perangkat penerus mengirim pesan Packet-In ke pengontrol SDN.

Pesan ini dipicu ketika perangkat perlu meneruskan paket tetapi tidak memiliki informasi yang diperlukan untuk membuat keputusan tersebut. Fungsi utamanya adalah untuk meminta panduan dari pengontrol SDN mengenai tindakan yang tepat untuk mengambil paket tertentu yang dimaksud.

Sebaliknya, pengontrol SDN memulai Peristiwa Packet-Out, yang mengirimkan pesan Packet-Out untuk menginstruksikan perangkat penerus untuk meneruskan paket tertentu ke port tujuan yang ditentukan. Peristiwa ini biasanya digunakan ketika menerapkan aturan penerusan ke masing-masing paket. Terakhir, Event Flow Modification (FlowMod) memiliki kemiripan dengan Event Packet-Out namun memiliki tujuan yang berbeda. Ini dirancang untuk menambah, memodifikasi, atau menghapus entri aliran dalam tabel aliran perangkat penerusan. Entri aliran ini menentukan bagaimana paket dengan karakteristik tertentu

harus ditangani, mencakup tindakan seperti meneruskan, menjatuhkan, atau memodifikasi paket. Event ini sangat cocok untuk menerapkan aturan penerusan ke beberapa paket yang memiliki karakteristik yang sama [26]. Perilaku ini memungkinkan switch untuk menyimpan tabel aliran yang akan digunakan untuk meneruskan lalu lintas ketika paket yang sama dengan tujuan yang sama datang. Pengontrol SDN modifikasi yang baru diusulkan menggunakan Ryu sebagai kerangka utama. Arsitektur yang diusulkan didasarkan pada layanan mikro seperti yang ditunjukkan pada Gambar 1. Dalam arsitektur ini, semua komponen dan aplikasi Ryu didekomposisi sebagai layanan mikro daripada digabungkan dalam fungsi lingkungan monolit.



GAMBAR 3. Arsitektur monolit pengontrol Ryu.

Arsitektur yang kami usulkan menerima tantangan dalam menggunakan arsitektur layanan mikro baru untuk menerapkan Pengontrol SDN yang ada. Pengontrol SDN yang ada biasanya diterapkan sebagai arsitektur monolitik, sehingga membatasi fleksibilitas penskalaannya ketika sub-fungsi yang lebih spesifik dalam pengontrol diperlukan seperti yang ditunjukkan pada Gambar. 2. Kami menggunakan Kerangka Ryu SDN karena ini adalah salah satu pengontrol yang Sesuai dengan 5G dan dapat dengan mudah diterapkan di Docker Containers [5]. Seperti yang ditunjukkan pada Gambar 1, setiap sub-fungsi di Ryu Controller akan dimasukkan ke dalam container dengan bantuan Docker dan diatur dengan Kubernetes.

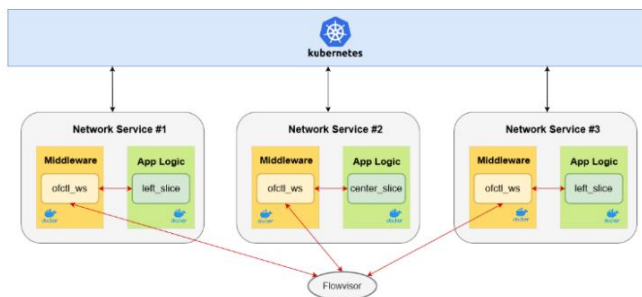
TABEL I
SPESIFIKASI MESIN

Node	CPU	RAM	Penyimpanan
Master Kubernet	2 Inti	2 GB	50 GB
Pekerja Kubernetes-1	2 Inti	2 GB	30 GB
Pekerja Kubernetes-2	2 Inti	2 GB	30 GB
Monolit Ryu	4 Inti	4 GB	30 GB
Pelindung aliran	2 Inti	2 GB	20 GB
Mininet	2 Inti	4 GB	20 GB

Hal ini membuat Pengontrol Ryu lebih fleksibel dan terukur seiring dengan meningkatnya permintaan lalu lintas yang meminta fungsi dan rute jaringan tertentu. Cluster dan lingkungan testbed, seperti ditunjukkan pada Gambar 3, terdiri dari 3 VM untuk Kubernetes Cluster dengan 1 Master dan 2 Worker, 1 VM untuk Ryu Monolith, 1 VM untuk Proxy Controller dalam hal ini kita menggunakan Flowvisor, dan terakhir 1 VM lainnya untuk Mininet. Spesifikasi masing-masing node dapat dilihat pada Tabel 1.

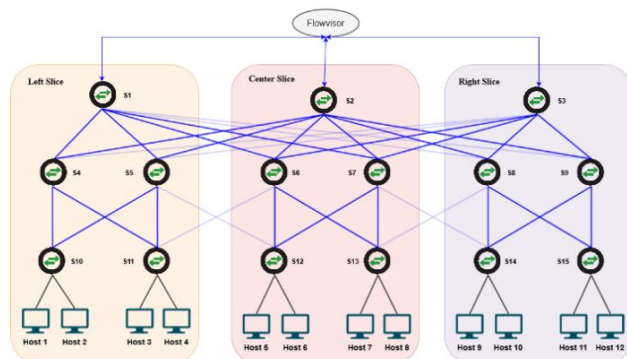
IV. SKENARIO KASUS PENGGUNAAN MSNWS

Pengontrol yang diusulkan diterapkan pada skenario yang berbeda. Subbagian berikut menyediakan dua kasus penggunaan pemotongan jaringan dan pemanfaatan sumber daya. Kami akan membandingkan kinerja MSNws dan Ryu Monolith.



GAMBAR 4. Arsitektur testbed MSNws.

Penerapannya akan dimasukkan ke dalam container dengan Docker dan diatur dengan Kubernetes, seperti yang ditunjukkan pada Gambar 4 dan dinyatakan dalam subbagian sebelumnya. Komponen middleware akan dihubungkan ke pengontrol proxy eksternal yaitu Flowvisor. Sedangkan untuk arsitektur testbed Ryu Monolith tidak menggunakan middleware di Ryu Monolith untuk terhubung ke Flowvisor, karena penerapan aplikasi Ryu akan menggunakan runtime handler Ryu OpenFlow default untuk menangani lalu lintas masuk yang dikirim dari Flowvisor.



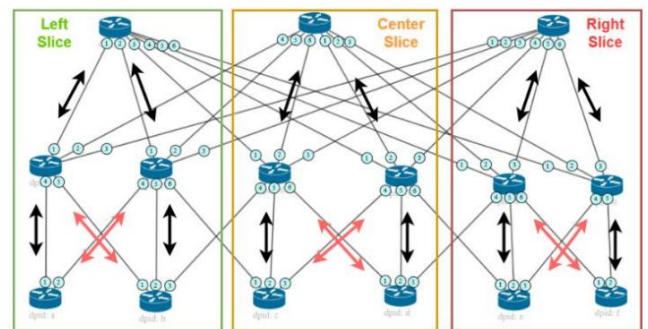
GAMBAR 5. Skenario pengujian Fat Tree.

Gambar 5 menunjukkan topologi Fat Tree yang digunakan di SDN untuk skenario ini, di mana berbagai layanan dibagi menjadi beberapa bagian yang disebut irisan, masing-masing dialokasikan dengan bandwidth yang disesuaikan. Terdiri dari 15 switch dan 12 host yang terhubung dalam pola yang mencerminkan jaringan multi-tenant. Setiap irisan akan terhubung ke Flowvisor, dan juga akan terhubung ke MSNws dan Ryu Monolith secara bergantian saat setiap arsitektur diuji, seperti yang ditunjukkan pada Gambar 4.

A. PENGIRIS JARINGAN

Kami menerapkan skenario pemotongan jaringan yang terdiri dari tiga irisan seperti yang ditunjukkan pada Gambar 5 yaitu irisan kiri, dikhususkan untuk video streaming dialokasikan bandwidth 3Mbps, dengan jalur pemotongan dari H1 hingga H4. Begitu pula dengan irisan tengahnya, sajikan

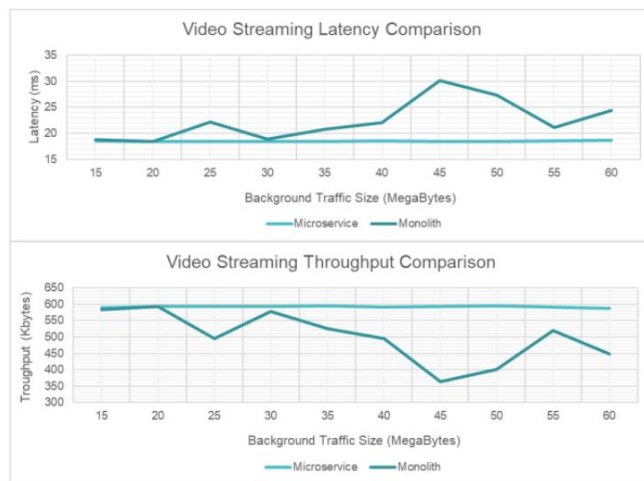
penelusuran web, dialokasikan bandwidth 1Mbps, mulai dari H5 hingga H8. Terakhir, irisan kanan, yang melayani VoIP, memiliki bandwidth 500Kbps, mencakup jalur pemotongan dari H9 ke H12. Skema alokasi bandwidth yang cermat dan jalur pemotongan yang dirancang dengan cermat ini memastikan pengoperasian berbagai layanan yang harmonis dalam jaringan multi-penyewa.



GAMBAR 6. Aturan aliran topologi Fat Tree.

Untuk membuat irisan jaringan yang terisolasi, kami menggunakan Flowvisor, yang dapat membuat ruang aliran untuk menentukan aturan aliran setiap irisan [27]. Seperti yang ditunjukkan pada Gambar 6, setiap saklar pada masing-masing irisan terhubung ke saklar masing-masing pada irisannya sendiri dan ke saklar milik irisan lainnya. Kami juga membuat tautan kompleks dalam topologi ini untuk menguji kredibilitas Flowvisor dalam mengisolasi jaringan dengan ruang alirannya konfigurasi. Misalnya, saklar atas pertama di irisan kiri akan dikonfigurasi sedemikian rupa sehingga lalu lintas pada irisan yang ditentukan hanya dapat mengalir melalui port 1 dan 2 dari antarmuka saklar tetapi tidak dapat mengalir melalui port 3, 4, 5, dan 6 karena port tersebut mengarah ke irisan yang berbeda. Contoh lainnya adalah saklar kiri bawah pada irisan kanan dengan konfigurasi ruang aliran yang ditentukan sehingga aliran hanya dapat melewati port 2 dan 3, namun tidak melalui port satu karena antarmuka ini mengarah ke irisan tengah. Begitu seterusnya, dengan switch lain yang terhubung ke port antarmuka.

Skenario pengujian akan dikelola dengan Mininet dan diotomatisasi dengan skrip Bash. Skrip Bash ini akan menjalankan setiap layanan streaming video, penelusuran web, VoIP, dan berbagai ukuran lalu lintas latar belakang secara bersamaan. Lalu lintas latar belakang dihasilkan menggunakan Iperf, dengan ukurannya yang meningkat secara bertahap. Kami menggunakan layanan streaming yang dibangun di VLC Media Player, jadi satu host akan bertindak sebagai server, dan host lainnya akan mengalirkan video dari host server. Selanjutnya, kami menggunakan situs web sederhana berdasarkan Python untuk layanan web, dan untuk VoIP, kami menggunakan DITG untuk menghasilkan lalu lintas. Kami juga membuat konfigurasi khusus tambahan di Pengontrol Ryu, terlepas dari MSNws atau Monolith, agar setiap layanan memiliki jalur aliran khusus yang berbeda dari lalu lintas latar belakangnya. Seperti yang ditunjukkan pada Gambar 6, kami mendefinisikan panah hitam sebagai jalur panjang dan panah merah sebagai jalur pendek. Lalu lintas setiap layanan utama, seperti streaming video, penelusuran web, dan VoIP pada setiap irisan, akan mengalir melalui jalur pendek untuk lalu lintasnya. Lalu lintas latar belakang akan menggunakan jalur panjang.



GAMBAR 7. Hasil QoS video streaming.

Gambar 7 menunjukkan bahwa MSNws memiliki kinerja latensi dan throughput yang lebih stabil. Stabilitas pada grafik tersebut berarti perubahan nilai dalam perubahan periodik pada kondisi yang berbeda [28]. Hal ini sangat penting dalam layanan streaming video, di mana pengguna menginginkan pengalaman yang lancar dan tanpa kerumitan. Dengan latensi yang stabil, pengguna dapat menikmati streaming video tanpa gangguan buffering yang mengganggu. Demikian pula, throughput yang tinggi menunjukkan kemampuan MSNws untuk mengirimkan data video secara efisien, sehingga memastikan video dapat diputar dengan baik.

Kemudian, terdapat perbedaan yang signifikan setelah meningkatkan lalu lintas latar belakang menjadi 25 MB dan kemudian hingga 60 MB, seperti yang ditunjukkan pada Gambar 7. MSNws masih berkinerja lebih konsisten dibandingkan Monolitik Ryu. Hal ini menunjukkan bahwa meskipun terjadi peningkatan beban trafik, MSNws masih dapat menjaga kestabilan kinerjanya. Sebaliknya, pengontrol monolitik Ryu mungkin lebih rentan terhadap fluktuasi lalu lintas latar belakang. Dalam konteks ini, MSNws dapat dianggap sebagai solusi yang lebih andal dalam menyediakan layanan streaming video berkualitas kepada pengguna, terutama ketika terjadi lonjakan lalu lintas yang tidak terduga.



GAMBAR 8. Hasil QoS penjelajahan web.

Lebih lanjut, analisis Gambar 8 mengungkapkan beberapa aspek menarik mengenai kinerja MSNws dengan arsitektur layanan mikro dibandingkan dengan pengontrol monolitik dalam konteks layanan web. Pertama, terdapat perbedaan latensi yang signifikan, di mana MSNw memiliki latensi lebih tinggi dibandingkan pengontrol monolitik. Hal ini menunjukkan bahwa pengontrol monolitik mungkin memiliki keuntungan dalam situasi ini dalam hal waktu respons. Namun, meskipun latensi MSNws lebih tinggi, kinerjanya tetap stabil, yang dapat dianggap sebagai aspek positif dari pengalaman pengguna.

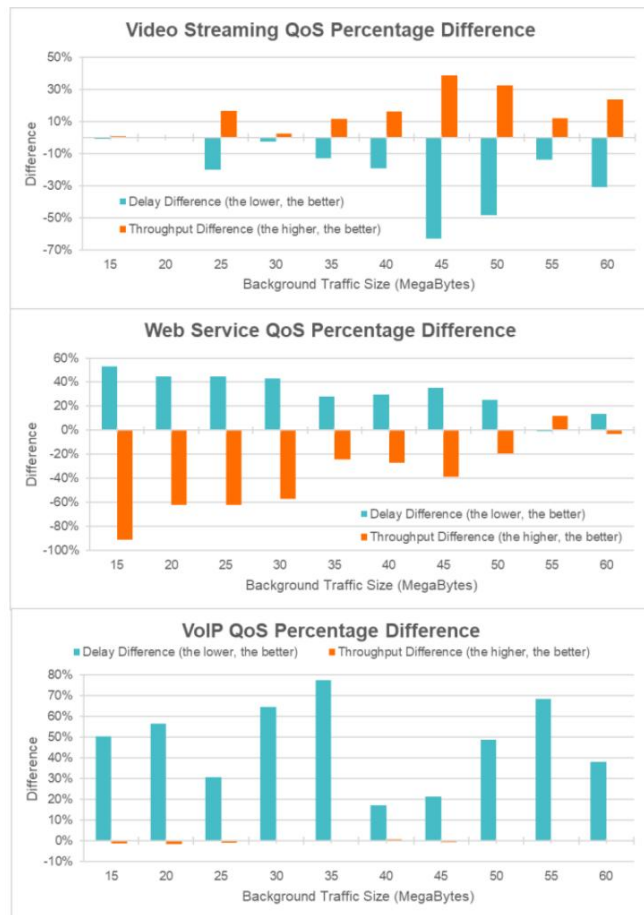
Kedua, throughput MSNws menunjukkan tingkat stabilitas yang lebih baik dibandingkan pengontrol monolitik. Artinya meskipun latensinya lebih tinggi, MSNws mampu menjaga konsistensi dalam pengiriman data atau layanan, yang dapat menjadi faktor penting dalam penjelajahan web yang memerlukan pengiriman data yang konsisten. Pada skala lalu lintas latar belakang 50-55 MB, keduanya tampak memiliki kinerja serupa, yang mungkin disebabkan oleh tingginya lalu lintas latar belakang yang berjalan dan pengaruh yang sama pada kedua pengontrol. Dalam konteks ini, penting juga untuk mempertimbangkan kebutuhan dan preferensi spesifik jaringan dan layanan dalam memilih pengontrol yang paling tepat. Jadi, meskipun MSNws memiliki latensi yang lebih tinggi, performanya tetap stabil, sementara pengontrol monolitik mungkin memiliki keunggulan dalam waktu respons.



GAMBAR 9. Hasil QoS VoIP.

Pada Gambar 9, Monolitik Ryu menunjukkan kinerja yang lebih baik dan lebih stabil dalam hal latensi dan throughput dibandingkan dengan MSNws. Analisis Pertama: Monolitik Ryu memiliki keunggulan yang cukup signifikan dalam hal latensi. Hal ini menunjukkan bahwa pengontrol monolitik dapat merespons permintaan dengan lebih cepat, yang mana hal ini sangat penting dalam layanan yang memerlukan waktu respons rendah, seperti VoIP atau layanan komunikasi real-time lainnya. Meski perbedaannya sedikit, namun kestabilan latensinya cukup baik. Kemudian, untuk analisis kedua, throughput Monolitik Ryu menunjukkan kinerja yang stabil. Meskipun tidak sepenuhnya dijelaskan dalam referensi ilmiah lainnya, kinerja throughput yang stabil dapat menunjukkan bahwa pengontrol monolitik dapat melakukannya.

menjaga konsistensi dalam penyampaian data atau layanan, yang penting dalam konteks layanan apa pun. Oleh karena itu, pengambilan keputusan antara kedua pengontrol harus didasarkan pada kebutuhan spesifik jaringan dan layanan yang akan digunakan, dengan mempertimbangkan trade-off antara kinerja dan stabilitas layanan.



GAMBAR 10. Perbedaan QoS

Gambar 10 mengilustrasikan variasi QoS di seluruh layanan yang diuji. Variasi ini menyoroti perbedaan yang signifikan untuk setiap skenario lalu lintas latar belakang. Dalam legenda, semakin baik menunjukkan kinerja yang unggul untuk MSNws. Grafik ini juga memberikan bukti mengenai arsitektur mana yang berkinerja lebih baik ketika menangani layanan tertentu dalam berbagai kondisi lalu lintas latar belakang. Misalnya, ketika memeriksa layanan Video Streaming, MSNws secara konsisten mengungguli monolit di hampir semua skenario lalu lintas latar belakang. Rata-rata, MSNws menunjukkan pengurangan penundaan sebesar 21,21% dan peningkatan throughput sebesar 15,49% dibandingkan dengan monolit.

Sebaliknya, Layanan Web dan VoIP menunjukkan kinerja yang lebih baik dengan arsitektur monolit. Layanan Web, rata-rata, menunjukkan penundaan 31,80% lebih rendah dan keluaran 37,31% lebih tinggi dibandingkan MSNws, sementara VoIP menunjukkan penurunan penundaan sebesar 47,32% dan sedikit peningkatan keluaran sebesar 0,45% dibandingkan dengan MSNws. Khususnya, nilai persentase ini tidak memperhitungkan nilai absolut penundaan dan throughput. Oleh karena itu, menilai persentase ini dalam

hubungannya dengan grafik hasil QoS sebelumnya disarankan untuk evaluasi komprehensif.

B. PENGGUNAAN SUMBER DAYA

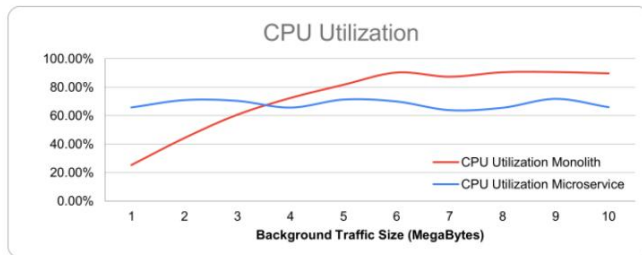
Pada subbagian ini, kami menguji penggunaan sumber daya dari MSNws yang kami usulkan dibandingkan dengan arsitektur monolit. Pengujian ini bertujuan untuk melihat penerapan arsitektur Pengontrol SDN mana yang lebih unggul dalam menangani sumber daya secara lebih efisien. Untuk skenario pengujian ini, kami membandingkan penggunaan sumber daya dari VM Kubernetes Cluster Worker dan VM Ryu Monolith karena ini lebih merupakan trade-off yang adil dibandingkan membandingkan antara Pod Kubernetes dan sebuah Instance.

Karena Kubernetes kami menggunakan teknologi containerisasi Docker dan sumber daya infrastruktur pengujian kami terbatas, ada beberapa catatan untuk lingkungan pengujian ini.

Pertama, container Docker berbagi sumber daya komputasi seperti CPU dan memori pada host, dan setiap container yang berjalan secara bersamaan akan bersaing untuk mendapatkan sumber daya bersama tersebut [29]. Oleh karena itu, untuk meminimalkan efek perilaku ini yang akan menurunkan kinerja container kami [29], kami menetapkan permintaan dan batasan untuk setiap penerapan Kubernetes aplikasi kami agar sumber daya terdistribusi secara merata. Rasio distribusi penerapan Kubernetes aplikasi pada irisan kiri, irisan tengah, dan irisan kanan adalah 2:1:1. Kami juga menghitung sisa sumber daya yang akan digunakan oleh host sekitar 10% dari seluruh sumber daya. Kedua, kami menggunakan Prometheus dan Grafana sebagai alat pemantauan eksternal untuk infrastruktur kami. Alat-alat ini diterapkan di luar infrastruktur kami dan akan membantu memantau secara akurat penggunaan sumber daya kami mengenai penggunaan CPU dan persentase penggunaan RAM karena pengujian kami akan ditekankan pada kemampuan maksimalnya. Terakhir, untuk skenario ini, kami menggunakan topologi yang sama seperti yang ditunjukkan pada Gambar 4.

Namun, kami tidak mengimplementasikan FlowMod di Pengontrol Ryu SDN, sehingga sakelar akan selalu meminta Aliran dari pengontrol, terus-menerus menekankan dan menggunakan sumber dayanya pada setiap lalu lintas masuk. Karena tujuan skenario ini adalah untuk melihat seberapa efisien penggunaan sumber daya MSNws kami dibandingkan dengan Monolith SDN Controller, kami melihat ini sebagai alternatif daripada lalu lintas latar belakang yang lebih luas yang juga akan menekankan sumber daya di instance Mininet kami.

Sebelum kita menjalankan skenario ini, kita perlu mengetahui keadaan awal mesin kita saat aplikasi diterapkan. Rata-rata penggunaan CPU dan penggunaan RAM pada VM Kubernetes Worker kami masing-masing adalah 2,80% dan 34,35%. Di sisi lain, rata-rata penggunaan CPU dan penggunaan RAM pada VM Ryu Monolith kami masing-masing adalah 0,41% dan 13,9%. Sumber daya tinggi yang sudah digunakan untuk VM Pekerja Kubernetes kami menunjukkan bahwa Kubernetes dan instance penerapannya telah mencadangkan sejumlah sumber daya. Penggunaan VM Ryu Monolith yang lebih rendah menunjukkan bahwa aplikasi disebarkan langsung ke host sebagai layanan tanpa perantara di antara keduanya.


GAMBAR 11. Pemanfaatan CPU

Gambar 11 menunjukkan hasil pemanfaatan CPU untuk skenario ini secara progresif dan dengan lalu lintas latar belakang yang meningkat secara bertahap. Grafik ini menunjukkan bahwa pengontrol layanan mikro melakukan penggunaan pemanfaatan CPU yang lebih stabil dibandingkan pengontrol monolit seiring dengan peningkatan ukuran lalu lintas latar belakang. Perilaku ini dimungkinkan karena kemampuan Kubernetes untuk membatasi dan mencadangkan sumber daya dari pod Kubernetes yang di-deploy, seperti yang disebutkan sebelumnya, dengan permintaan dan batasan penerapan. Setelah lalu lintas latar belakang 4 hingga 6 MB, MSNws kami bisa mengungguli Ryu Monolith, dengan rata-rata penggunaan CPU sekitar 18% lebih efisien. Selain itu, jika kita mempertimbangkan keadaan awal penggunaan CPU sebelum skenario dijalankan, perbandingan peningkatan puncak penggunaan CPU Ryu Monolith adalah 21% lebih tinggi dibandingkan MSNws. Ini berarti Ryu Monolith memiliki penggunaan CPU yang lebih tinggi pada kemampuan penggunaan puncaknya dibandingkan MSNws.

Di sisi lain, penggunaan RAM untuk kedua arsitektur cukup tidak meningkat secara signifikan setelah skenario berjalan. Bahkan ketika kita membandingkan hasil dari setiap lalu lintas latar belakang yang berbeda, peningkatannya tidak signifikan karena rata-rata sekitar 0,4% untuk Ryu Monolith dan 2,2% untuk MSNws. Secara keseluruhan, ekspektasi penggunaan sumber daya MSNws dibandingkan Ryu Monolith bisa lebih efisien dalam pemanfaatan CPU tetapi dengan beberapa persyaratan agar RAM dapat digunakan lebih banyak pada keadaan awal dan peningkatan lebih tinggi sekitar 1,8%.

V. PERMASALAHAN TERBUKA DAN KESIMPULAN

Secara keseluruhan, penelitian ini memberikan pemahaman yang lebih mendalam tentang peran arsitektur layanan mikro dalam mengelola jaringan dengan lalu lintas tinggi, khususnya dalam konteks jaringan perusahaan. Hasil penelitian menunjukkan bahwa SDN Controller berbasis microservices mempunyai keunggulan dalam menjaga kestabilan performa jaringan dan penggunaan resource CPU. Namun, perlu dicatat bahwa pengontrol layanan mikro menunjukkan kinerja yang lebih rendah dan lebih tidak stabil dibandingkan dengan pengontrol monolitik ketika dihadapkan dengan layanan dengan aliran data konstan, seperti VoIP. Oleh karena itu, pilihan antara kedua jenis pengontrol harus didasarkan pada kebutuhan jaringan tertentu, dengan mempertimbangkan trade-off antara kinerja dan stabilitas. Penelitian ini juga membuka pintu bagi penelitian lebih lanjut dalam mengoptimalkan penggunaan arsitektur layanan mikro di lingkungan jaringan yang beragam. Pengembangan lebih lanjut dari arsitektur ini dapat meningkatkan kinerja di berbagai layanan sekaligus menjaga fleksibilitas yang diberikan oleh pendekatan ini. Dengan begitu, jaringan yang efisien dan dapat diandalkan dapat tercipta

tetap terjaga, bahkan menghadapi tantangan lalu lintas jaringan yang dinamis dan beragam.

Sistem yang diusulkan (MSNws) dapat ditingkatkan untuk mengembangkan komponen perangkat lunak atau logika aplikasi yang lebih dinamis. Dalam konteks ini, dinamis berarti perangkat lunak mengikuti praktik terbaik menggunakan layanan pendukung seperti database untuk menyimpan alamat MAC dan data topologi lainnya. Dengan menyimpan data topologi dalam database, logika aplikasi dapat menjadi lebih efisien dan terukur karena data topologi tidak perlu diduplikasi saat melakukan upgrade komponen logika aplikasi. Hal ini karena komponen logika aplikasi tidak lagi memperluas data topologi penyimpanan. Selain itu, penelitian ini diharapkan dapat mempertimbangkan penggunaan mekanisme penyimpanan persisten seperti Persistent Volumes (PV) Kubernetes untuk layanan database. Dengan melakukan hal ini, data penting seperti data topologi akan lebih aman dan terlindungi serta dapat diakses secara konsisten bahkan dalam situasi yang memerlukan skalabilitas atau pemulihan setelah kegagalan. Dengan mengintegrasikan PV ke dalam arsitektur, MSNws dapat meningkatkan keandalan dan ketersediaan sistem secara keseluruhan, yang sangat penting dalam memastikan kinerja optimal dalam manajemen jaringan yang kompleks.

PENGAKUAN

Kami mengucapkan terima kasih yang sebesar-besarnya atas dukungan tak ternilai yang diberikan selama penelitian ini. Dukungan ini dengan mudah hati diberikan melalui infrastruktur penelitian Adaptive Network Laboratory, sebuah proyek yang terlaksana melalui pendanaan dan komitmen Telkom University.

REFERENSI

- [1] MB Jimenez, D. Fernandez, JE Rivadeneira, L. Bellido, dan A. Cardenas, "Survei Masalah Keamanan Utama dan Solusi untuk Arsitektur SDN," *IEEE Access*, vol. 9, hal.122016–122038, 2021, doi: 10.1109/ACCESS.2021.3109564.
- [2] MU Younus, S. ul Islam, I. Ali, S. Khan, dan M. K. Khan, "Survei tentang jaringan yang ditentukan perangkat lunak memungkinkan bangunan pintar: Arsitektur, tantangan dan kasus penggunaan," *Jurnal Aplikasi Jaringan dan Komputer*, vol. 137, hlm. 62–77, Juli 2019, doi: 10.1016/j.jnca.2019.04.002.
- [3] F. Hu, Q. Hao, dan K. Bao, "Survei tentang Jaringan yang Ditentukan Perangkat Lunak dan OpenFlow: Dari Konsep hingga Implementasi," *Survei & Tutorial Komunikasi IEEE*, vol. 16, tidak. 4, hal.2181–2206, 2014, doi: 10.1109/COMST.2014.2326417.
- [4] L. Nóvoa, V. Tavares, C. Nahum, S. Lins, dan A. Klautau, "Implementasi middleware untuk Pengontrol RYU SDN untuk mengelola sakelar dalam skenario C-RAN," dalam *Anais do XLVIII Seminário Integrado de Software e Hardware (SEMISH 2021)*, Sociedade Brasileira de Computação - SBC, Juli 2021, hlm. 19–29. doi: 10.5753/semish.2021.15803.
- [5] ST Arzo dkk., "MSN: Kerangka Kerja untuk Desain dan Evaluasi Berbasis Layanan Mikro

- sdN Controller," *Jurnal Manajemen Jaringan dan Sistem*, vol. 30, tidak. 1, hal. 19 Januari 2022, doi: 10.1007/s10922-021-09631-7.
- [6] Samsung Electronics Corporation, "Cloud Native 5G Core Samsung 5G Core Vol.2," Januari 2020. [On line]. Tersedia: <http://gsacom.com/technology/5g/>
- [7] HV Netto, LC Lung, M. Correia, AF Luiz, dan LM Sá de Souza, "Replikasi mesin status dalam container yang dikelola oleh Kubernetes," *Jurnal Arsitektur Sistem*, vol. 73, hlm. 53–59, Februari 2017, doi: 10.1016/j.sysarc.2016.12.007.
- [8] IM Al Jawarneh dkk., "Container Orchestration Engines: A Thorough Functional and Performance Comparison," dalam *ICC 2019 - 2019 IEEE International Conference on Communications (ICC)*, IEEE, Mei 2019, hlm. 1–6. doi: 10.1109/ICC.2019.8762053.
- [9] L.-M. Tufeanu, A. Mars, M.-C. Vochin, C.-L. Paraschiv, dan FY Li, "Membangun Jaringan 5G SA dalam Kontainer Sumber Terbuka melalui Docker dan Kubernetes," pada *Simposium Internasional ke-25 tentang Komunikasi Multimedia Pribadi Nirkabel (WPMC) tahun 2022*, IEEE, Oktober 2022, hlm. doi: 10.1109/WPMC55625.2022.10014753.
- [10] A. Pacini, D. Scano, L. Valcarenghi, A. Sgambelluri, dan A. Giorgetti, "Mengaktifkan sinkronisasi hierarki berbasis peristiwa di cluster SDN ONOS," pada *Konferensi IEEE 2022 tentang Virtualisasi Fungsi Jaringan dan Jaringan yang Ditentukan Perangkat Lunak (NFV-SDN)*, IEEE, November 2022, hlm. 93. doi: 10.1109/NFV-SDN56302.2022.9974775.
- [11] H. Donertasli dan Z. Altan, "Vendor Independent SDN Architecture Solution," pada *Konferensi Internasional Ilmu dan Teknik Komputer (UBMK) ke-5 tahun 2020*, IEEE, September 2020, hlm. doi: 10.1109/UBMK50275.2020.9219421.
- [12] Z. Awada, K. Boulous, M. El-Helou, K. Khawam, dan S. Lahoud, "Distributed multi-tenant RAN slicing in 5G network," *Wireless Networks*, vol. 28, tidak. 7, hlm. 3185–3198, Oktober 2022, doi: 10.1007/s11276-022-03023-8.
- [13] A. Rafiq, A. Mehmood, dan W.-C. Lagu, "Pengirisan Berbasis Niat antar Kontainer di Jaringan Overlay SDN," *Jurnal Komunikasi*, hal.237–244, 2020, doi: 10.12720/jcm.15.3.237-244.
- [14] X. Zhou, R. Li, T. Chen, dan H. Zhang, "Pengirisan jaringan sebagai layanan: memungkinkan jaringan seluler yang ditentukan oleh perangkat lunak milik perusahaan," *IEEE Communications Magazine*, vol. 54, tidak. 7, hal.146–153, Juli 2016, doi: 10.1109/MCOM.2016.7509393.
- [15] N.Zhang, Y.-F. Liu, H. Farmanbar, T.-H. Chang, M.Hong, dan Z.-Q. Luo, "Pemotongan Jaringan untuk Jaringan Berorientasi Layanan di Bawah Kendala Sumber Daya," *Jurnal IEEE tentang Area Terpilih di Komunikasi*, jilid. 35, tidak. 11, hal. 2512–2521, November 2017, doi: 10.1109/JSAC.2017.2760147.
- [16] AM Escolar, JM Alcaraz-Calero, P. Salva-Garcia, JB Bernabe, dan Q. Wang, "Pemotongan Jaringan Adaptif dalam Jaringan IoT 5G Multi-Penyewa," *IEEE Access*, vol. 9, hal. 14048–14069, 2021, doi: 10.1109/ACCESS.2021.3051940.
- [17] CV Nahum dkk., "Uji Kecerdasan Buatan Terhubung 5G di Jaringan Virtual," *Akses IEEE*, jilid. 8, hal. 223202–223213, 2020, doi: 10.1109/ACCESS.2020.3043876.
- [18] Yayasan Jaringan Terbuka, "onosproject." Diakses: 03 September 2023. [Online]. Tersedia: <https://docs.onosproject.org/>
- [19] S. Yun, J. Park, H. Kim, dan W.-T. Kim, "Importance-aware SDN Control Mechanism for Real-time Data Distribution Services," pada *Konferensi Internasional tentang Konvergensi Teknologi Informasi dan Komunikasi (ICTC) 2018*, IEEE, Oktober 2018, hlm. 1113–1118. doi: 10.1109/ICTC.2018.8539690.
- [20] S. Bhowmik, "Memanfaatkan kekuatan jaringan yang ditentukan perangkat lunak untuk middleware komunikasi berkinerja tinggi," dalam *Prosiding Lokakarya Internasional Kedua tentang Middleware Aktif pada Perangkat Keras Modern*, New York, NY, AS: ACM, Desember 2017, hal. 14–14. doi: 10.1145/3155889.3155894.
- [21] R. Sherwood dkk., "FlowVisor: Lapisan Virtualisasi Jaringan," Oktober 2009.
- [22] AM Potdar, N. DG, S. Kengond, dan MM Mulla, "Evaluasi Kinerja Kontainer Docker dan Mesin Virtual," *Procedia Comput Sci*, vol. 171, hlm. 1419–1428, 2020, doi: 10.1016/j.procs.2020.04.152.
- [23] M. Pratap Yadav, N. Pal, dan D. Kumar Yadav, "Pendekatan formal untuk penerapan container Docker," *Komputasi Setuju*, vol. 33, tidak. 20 Oktober 2021, doi: 10.1002/cpe.6364.
- [24] N. Nguyen dan T. Kim, "Menuju Penyeimbangan Beban yang Sangat Skalabel di Cluster Kubernetes," *Majalah Komunikasi IEEE*, vol. 58, tidak. 7, hal.78–83, Juli 2020, doi: 10.1109/MCOM.001.1900660.
- [25] flannel-io, "flannel." Diakses: 03 September 2023. [On line]. Tersedia: <https://github.com/flannel-io/flannel>
- [26] Open Networking Foundation, "Spesifikasi OpenFlow Switch Versi 1.5.1," 2015. [Online]. Tersedia: <http://www.opennetworking.org>
- [27] T. Alharbi dan M. Portmann, "Keamanan (Dalam) Virtualisasi dalam Jaringan yang Ditentukan Perangkat Lunak," *Akses IEEE*, jilid. PP, hal. 1, September 2019, doi: 10.1109/ACCESS.2019.2918101.
- [28] N. Xie, W. Tan, X. Zheng, L. Zhao, L. Huang, dan Y. Sun, "Pendekatan dua fase yang efisien untuk komposisi layanan sadar kolaborasi yang andal di

manufaktur cloud," *J Ind Inf Integr*, vol. 23, hal.
100211, September 2021, doi: 10.1016/j.jii.2021.100211.

- [29] L. Cai, Y. Qi, W. Wei, dan J. Li, "Meningkatkan
Penggunaan Sumber Daya Kontainer melalui
Penyetelan Otomatis Parameter Sumber Daya Kontainer,"
IEEE Access, vol. 7, hal. 108530–108541, 2019, doi:
10.1109/ACCESS.2019.2927279.